

INFORME TÉCNICO

LogiSmart Web

Transformación del script monolítico `logiuncodigo.py` en una aplicación modular con persistencia en MongoDB, LLM local (Ollama) e interfaz gráfica interactiva

Stack

Flask · Bootstrap 5 · Chart.js · MongoDB · Ollama (llama3 8B) · pydantic

Verificación: 20/20 pruebas unitarias · 10 páginas con respuesta 200

1. Arquitectura por capas

```
ejercicio2_logismart/
├── run.py                # punto de entrada (http://127.0.0.1:5000)
├── config.py             # configuración global / variables de entorno
├── datos/
│   ├── correos_etiquetados.json # dataset de 34 correos etiquetados a mano
│   └── local_fallback.json      # almacén de respaldo (se genera solo)
├── app/
│   ├── __init__.py          # app factory + carga de dataset + seed
│   ├── db.py                # CAPA DE PERSISTENCIA (Mongo / respaldo local)
│   ├── reglas.py            # motor de reglas proposicionales + tablas + análisis
│   ├── esquemas.py          # contratos pydantic (API y JSON del LLM)
│   ├── llm.py               # cliente Ollama (disponibilidad, modelos, latencia)
│   ├── clasificador.py      # reglas + LLM + fusión híbrida + evaluación
│   ├── asistente.py         # RAG: recuperación -> contexto -> respuesta citada
│   ├── seed.py              # datos de demostración
│   ├── reportes.py          # exportación CSV / JSON
│   └── rutas/               # CAPA API REST (blueprints)
│       ├── vistas.py        # páginas HTML
│       ├── api_nucleo.py     # estado, dashboard, config, seed, reportes
│       ├── api_accesos.py    # camiones CRUD, evaluación de acceso, reglas
│       ├── api_incidentes.py  # clasificar, CRUD incidentes, evaluación asíncrona
│       ├── api_riesgos.py     # CRUD matriz de riesgos + resumen
│       └── api_asistente.py   # chat RAG + historial
│   └── templates/           # CAPA DE PRESENTACIÓN (Jinja + Bootstrap 5)
│       └── static/ (css, js) # helpers compartidos del frontend
└── tests/test_logismart.py  # 20 pruebas unitarias
```

El script original sigue vivo en los módulos equivalentes:

Sección original	Módulo nuevo
PEAS	reglas.PREMISAS / documentación (modelo ya aplicado al dominio)
evaluar_camion, tablas de verdad	app/reglas.py
clasificar_incidente, extraer_datos	app/clasificador.py (modo <i>reglas</i>)
EvaluadorRiesgosIA	app/rutas/api_riesgos.py + app/esquemas.nivel_riesgo

2. Persistencia (MongoDB)

Cadena por defecto mongodb://localhost:27017, base logismart, sobrescribible con MONGO_URI (apunta también a Atlas). Si el *ping* falla al arrancar —o al reconectar desde Configuración— se usa un almacén JSON local con la misma interfaz de colección (la GUI muestra el motor activo en la barra lateral).

Esquema de colecciones

Colección	Documento
camiones	placa, camion_id, empresa, autorizacion, certificacion_vigente, certificacion_dias_restantes, creado_en
accesos	camion_id, placa, premisas P Q R S T V, salidas A E H W, decision, semaforo, explicacion[], operador, timestamp
incidentes	remitente, asunto, cuerpo, clasificacion{categoria, prioridad, fuente, resumen, palabras_clave, detalle}, datos_extraidos{placa, camion_id, peso_reportado_kg, ubicacion}, estado (nuevo/en_atencion/cerrado), requiere_revision_humana, timestamp, historial[]
riesgos_eticos	modulo, descripcion, categoria, probabilidad, impacto, mitigacion, probabilidad_residual, impacto_residual, historico[]
evaluaciones_llm	run_id, modo, modelo, prompt, respuesta{exactitudes, matriz}, latencia, coincidio_reglas, total_correos, timestamp
conversaciones	sid, pregunta, respuesta, fuentes[], modo, timestamp
configuracion	modelo, umbral_confianza, simulacion_correo, operador

Agregación implementada

Incidentes por categoría y semana ISO con pipeline real de MongoDB (`$group + $isoWeek + $dateFromString`); la gráfica del panel consume el endpoint `GET /api/incidentes/agregacion`. El modo local calcula el mismo resultado en Python para no romper la demostración.

CRUD completo desde la GUI: camiones, incidentes (incluye corrección manual de categoría/prioridad y cambio de estado con historial) y riesgos.

3. Motor de reglas

Reglas conservadas

- **A** (acceso estándar) = $P \wedge S \wedge \neg Q$
- **E** (inspección especial) = $P \wedge (R \vee Q)$

Reglas nuevas (justificadas, con tabla de verdad)

Regla	Fórmula	Justificación	Tabla (casos V)
H — Retención por horario	$R \wedge T$	Las normas de materiales peligrosos restringen su circulación nocturna; si $R \wedge T$ el camión se retiene aunque A o E valgan.	H vale en 1/4 combinaciones ($R=T=V$)
W — Certificación por vencer	$S \wedge V$	Una certificación con <30 días de vigencia es riesgo regulatorio: el acceso se permite pero genera alerta al operador.	W vale en 1/4 combinaciones ($S=V=V$)

Precedencia de la decisión: $\neg P \rightarrow \text{denegado} > H \rightarrow \text{retenido} > E \rightarrow \text{inspección} > A \rightarrow \text{estándar} > \text{resto} \rightarrow \text{denegado}$. Cada acceso guarda la **explicación paso a paso** (qué premisa causó qué) en la colección accesos.

Reto opcional resuelto (GET /api/reglas/analisis)

Sobre las 64 combinaciones: **ninguna regla es redundante** con otra, y no hay contradicciones irresolubles; se detectan y reportan solapamientos que el orden de precedencia resuelve:

Solapamiento	Casos
A $\wedge$ E	4
E $\wedge$ H	8
A $\wedge$ H	2

4. Clasificador híbrido

Contrato JSON del LLM (validado con pydantic)

```
{
  "categoria": "materiales_peligrosos|sobrepeso|acceso_no_autorizado|
               falla_hardware|falla_software|somnolencia_conductor|otro",
  "prioridad": "baja|media|alta|critica",
  "entidades": {
    "placa": "...|null",
    "camion_id": "...|null",
    "peso_reportado_kg": 0.0,
    "ubicacion": "...|null"
  },
  "resumen": "una frase"
}
```

El prompt completo está en `app/clasificador.PROMPT_CLASIFICADOR`: pide JSON estricto (`format="json"` en Ollama), define cada categoría y las reglas de prioridad. Si el JSON no valida, se reintentará con un mensaje corrector (máx. 2 intentos); si persiste, **cae al clasificador por reglas** (plan de respaldo) y lo reporta en `error_llm`.

Fusión

- Prioridad final = la **más alta** de ambos («ante la duda, seguridad»).
- Categoría = la del clasificador que reportó mayor prioridad; en empate, la del LLM.
- Si discrepan en categoría o prioridad → `requiere_revision_humana = true` (bandera visible en la bandeja).

Resultados del experimento (dataset de 34 correos)

Ejecutado desde `/evaluacion` sobre `datos/correos_etiquetados.json` (incluye 2 correos con ortografía informal a propósito):

Modo	Exactitud categoría	Exactitud prioridad	Latencia media
Reglas	88.2 % (30/34)	70.6 %	< 1 ms
LLM (llama3 8B, CPU)	medido por GUI	medido por GUI	~90 s/correo
Híbrido	medido por GUI	medido por GUI	~90 s/correo

Los errores típicos de reglas son correos informales/ambiguos (p. ej. el id 34 con faltas intencionales) — exactamente el riesgo de sesgo documentado en la matriz. La latencia muestra el costo real de la inferencia local en CPU: las reglas son prácticamente instantáneas y el LLM aporta comprensión semántica a costa de tiempo; el híbrido conserva seguridad (reglas) con cobertura (LLM).

5. Asistente explicativo (RAG)

Patrón: **consulta primero, contexto después**. `app/asistente.py` detecta referencias (CAM-###, placas) e intenciones (incidentes abiertos, accesos, riesgos), recupera los documentos y construye un contexto etiquetado [`coleccion#id`]. El prompt (`PROMPT_ASISTENTE`) obliga a responder **solo** con el contexto y a citar; sin datos → frase fija «No tengo información registrada sobre eso.»; sin LLM → respaldo determinista con los registros.

Ejemplo real verificado: «¿Por qué CAM-102 fue enviado a inspección?» → el asistente citó [accesos#...] y explicó $E = P \wedge (R \vee Q) = V$ usando el razonamiento guardado en la bitácora.

6. Matriz de riesgos éticos

Sembrada con los riesgos de **esta** implementación (probabilidad × impacto 1–5, puntaje residual tras mitigación):

Módulo	Riesgo	P×I	Nivel	Residual
Clasificador híbrido	Alucinaciones del LLM ante correos ambiguos	4×4=16	alto	6
Clasificador por reglas	Sesgo ante ortografía informal/regionalismos	4×3=12	alto	6
Gestión de camiones	Privacidad de datos del conductor	3×5=15	alto	8
Sistema completo	Dependencia excesiva de la automatización	3×4=12	alto	6
Asistente RAG	Respuestas inventadas sin datos	3×4=12	alto	6
Motor de reglas	Regla nueva mal configurada bloquea accesos	2×4=8	medio	3

La GUI (/riesgos) muestra KPIs por nivel, la **matriz de calor 5×5**, la gráfica **inherente vs residual** y el CRUD con historial de cambios.

7. Interfaz y usabilidad

- **Panel:** KPIs + 3 gráficas (decisión, categoría, semana) con filtro de fechas.
- **Control de acceso:** búsqueda por placa (prellena premisas desde la ficha), semáforo, explicación y bitácora.
- **Simulador:** 6 interruptores con salida A/E/H/W en vivo + tablas + análisis.
- **Incidentes:** pegar correo → clasificar (modo elegible) → editar → guardar; tabla con cambio de estado, filtros y detalle con historial.
- **Evaluación:** job asíncrono con barra de progreso, matrices y detalle por correo.
- **Asistente:** chat con historial persistente y chips de fuentes.
- **Reportes:** CSV/JSON del servidor + PDF en cliente (jsPDF).
- **Configuración:** estado de MongoDB con reconexión, modelo Ollama, umbrales, modo simulación y resembrado demo.
- **Usabilidad:** validación pydantic + mensajes claros en toasts, indicadores de carga en todas las llamadas al LLM, estados de error amigables y diseño consistente (sidebar, badges, componentes Bootstrap).

8. Verificación

- `python -m unittest discover -s tests -v` → **20/20 OK** (reglas originales+nuevas, explicación, análisis, clasificador por reglas, respaldo sin LLM, esquemas pydantic, matcher del almacén local, dataset ≥30).
- Verificación end-to-end: las 10 páginas responden 200; acceso evaluado y persistido; clasificación LLM real (llama3) correcta con entidades; RAG con citas reales; «no tengo información» sin datos; evaluación y agregación OK.